

Kapitel 3: Grundlagen der Programmierung

In diesem Kapitel geht es um die ersten grundlegenden Bausteine der Programmierung mit Python.

Wir betrachten Kommentare, Syntax-Highlighting, Variablen, Datentypen, Ein- und Ausgaben sowie einfache Operatoren.

Viele dieser Begriffe wirken am Anfang sehr technisch.

Wichtig ist deshalb: Ihr müsst nicht alles sofort auswendig können. Entscheidend ist, dass ihr versteht, wofür die einzelnen Bausteine gebraucht werden und dass ihr sie in einfachen Programmen anwenden könnt.

3.1. Kommentare

Beim Programmieren ist es sinnvoll, den eigenen Code an wichtigen Stellen zu kommentieren.

Kommentare werden bei der Ausführung eines Programms ignoriert. Sie sind also nicht für den Computer gedacht, sondern für Menschen, die den Code lesen.

Kommentare können zum Beispiel erklären, warum eine bestimmte Anweisung verwendet wird, was eine Variable bedeutet oder welcher Gedanke hinter einem Programmabschnitt steckt. Das hilft besonders dann, wenn man den eigenen Code später noch einmal verstehen möchte.

In Python beginnt ein einzeliger Kommentar mit dem Zeichen `#`.

```
# Das ist ein Kommentar.
```

Dass es sich um einen Kommentar handelt, erkennt man in Visual Studio Code auch am **Syntax-Highlighting**.

Kommentare werden anders eingefärbt als ausführbarer Programmcode.

3.2. Syntax-Highlighting

Syntax-Highlighting ist eine Funktion von Texteditoren und Entwicklungsumgebungen wie Visual Studio Code. Dabei werden verschiedene Bestandteile eines Programms farblich hervorgehoben.

Das macht Quellcode besser lesbar und hilft dabei, Strukturen und Fehler schneller zu erkennen. Wenn beispielsweise eine Zeichenkette nicht richtig geschlossen wurde, fällt das durch die veränderte Farbgebung oft sofort auf.

Typischerweise werden folgende Elemente hervorgehoben:

- **Schlüsselwörter:** feste Wörter der Programmiersprache, zum Beispiel `if`, `else` oder `while`
- **Kommentare:** Hinweise im Code, die nicht ausgeführt werden
- **Zeichenketten:** Text in Anführungszeichen, zum Beispiel `'Hallo Welt'`
- **Funktionen:** Befehle wie `print()` oder `input()`
- **Variablen:** Namen, unter denen Werte gespeichert werden
- **Zahlen, Operatoren und Symbole:** zum Beispiel `42`, `+`, `-`, `*` oder `=`

Zusammengefasst: Syntax-Highlighting verändert nicht, was ein Programm tut. Es hilft aber Menschen dabei, Programmcode leichter zu lesen und zu verstehen.



3.3. Eingabe, Verarbeitung, Ausgabe und Speicherung (EVAS-Prinzip)

Viele Programme lassen sich mit dem **EVAS-Prinzip** beschreiben:

- **Eingabe:** Daten werden eingegeben, zum Beispiel über die Tastatur.
- **Verarbeitung:** Die Daten werden vom Programm verarbeitet.
- **Ausgabe:** Das Programm gibt ein Ergebnis aus.
- **Speicherung:** Daten werden in Variablen oder Dateien gespeichert.

Ein einfaches Beispiel ist ein Taschenrechner: Man gibt Zahlen ein, das Programm berechnet ein Ergebnis und gibt dieses Ergebnis anschließend aus. Währenddessen müssen die eingegebenen Zahlen und das Ergebnis gespeichert werden.

Variablen

Variablen gehören zu den wichtigsten Elementen in der Programmierung. Sie dienen dazu, **Werte zu speichern**.

Man kann sich eine Variable wie eine beschriftete Box vorstellen: In der Box liegt ein Wert, und über den Namen der Box kann das Programm später wieder auf diesen Wert zugreifen.

In Python entsteht eine Variable durch eine **Zuweisung**. Dabei wird einem Variablennamen ein Wert zugewiesen. Die Variable `ich_bin_eine_variable` erhält im folgenden Beispiel den Wert `42`.

```
ich_bin_eine_variable = 42
```

Eine Zuweisung folgt immer diesem Grundschemata:

```
variable = ausdruck
```

Der Ausdruck rechts vom Gleichheitszeichen wird zuerst ausgewertet.

Danach wird das **Ergebnis** im Variablennamen **links** vom Gleichheitszeichen gespeichert.

Die Zuweisung erfolgt also gedanklich **von rechts nach links**.

Wichtig: Das Gleichheitszeichen `=` bedeutet in Python nicht dasselbe wie in der Mathematik. In Python steht `=` für eine Zuweisung. Für einen Vergleich auf Gleichheit verwendet man später den Operator `==`.

Gute Variablennamen sind wichtig, weil sie den Code verständlicher machen.

Schlechter Variablenname: `sn = 'Mustermann'`

Besserer Variablenname: `schueler_nachname = 'Mustermann'`

In Python verwendet man für Variablennamen üblicherweise den sogenannten **Snake Case**. Dabei werden alle Wörter klein geschrieben und durch Unterstriche verbunden. Sonderzeichen dürfen nicht verwendet werden, und Variablennamen dürfen nicht mit einer Zahl beginnen.

Beispiele für gültige Variablennamen:

- `alter`
- `vorname`
- `schueler_nachname`
- `anzahl_punkte`

In anderen Programmiersprachen gibt es auch andere Schreibweisen. Eine häufige Schreibweise ist der **camelCase**.

```
schuelerNachname = 'Mustermann'
```

Beim **camelCase** werden mehrere Wörter ohne Leerzeichen zusammengeschrieben. Ab dem zweiten Wort beginnt jedes neue Wort mit einem Großbuchstaben. Beispiele aus dem Alltag sind `YouTube`, `iPhone` oder `eBay`.

In diesem Kurs verwenden wir für Python-Programme konsequent die **Snake-Case-Schreibweise**.

Datentypen

Eine Variable speichert immer einen Wert. Jeder Wert besitzt einen **Datentyp**. Der Datentyp legt fest, welche Art von Daten gespeichert wird und welche Operationen damit sinnvoll möglich sind.

Für den Einstieg unterscheiden wir zunächst diese grundlegenden Datentypen:

Bezeichnung	Kurzform	Bedeutung	Beispiel(e)	Hinweis
integer	int	ganze Zahl	3 , 42 , -7	keine Nachkommastellen
float	float	Dezimalzahl / Gleitkommazahl	3.1 , 4.2	In Python wird ein Dezimalpunkt verwendet, kein Dezimalkomma.
string	str	Zeichenkette / Text	'Hallo' , 'Test123'	Zeichenketten stehen in einfachen oder doppelten Anführungszeichen.
boolean	bool	Wahrheitswert	True , False	Die Anfangsbuchstaben werden groß geschrieben.
none	NoneType	kein Wert / nichts	None	Der Wert <code>None</code> wird mit großem Anfangsbuchstaben geschrieben.

Mit der Funktion `type()` kann man prüfen, welchen Datentyp ein Wert oder eine Variable besitzt. Führt die folgenden Code-Blöcke aus und betrachtet die Ausgabe.

```
beispiel_zahl = 420.187
type(beispiel_zahl)
```

```
beispiel_text = 'BvC'
type(beispiel_text)
```

Ausgaben

Damit ein Python-Programm etwas ausgibt, verwendet man die Funktion `print()` .

```
test = 'Ich bin ein Test'
print(test)
print('--- erfolgreich ---')
```

Mit `print()` kann man nicht nur eine einzelne Variable ausgeben, sondern auch mehrere Texte und Variablen miteinander kombinieren. Eine einfache Möglichkeit ist die Trennung durch Kommas.

```
variable_eins = 'No'
variable_zwei = 'mercy'
print(variable_eins, 'Backup,', 'no', variable_zwei, '.')
```

Dabei fällt auf, dass Python zwischen den einzelnen Bestandteilen automatisch Leerzeichen einfügt. Dadurch erscheint im Beispiel auch vor dem Punkt ein Leerzeichen.

Zur besseren Verbindung von Variablen und Text eignen sich **F-Strings**. Dabei steht vor der Zeichenkette ein kleines `f` . Variablen, die eingesetzt werden sollen, schreibt man in geschweifte Klammern.

```
ergebnis = 42
print(f'Die Antwort auf die Frage nach dem Leben, dem Universum und dem ganzen Rest ist: {ergebnis}')
```

Eingaben

Damit Nutzerinnen und Nutzer Daten eingeben können, verwendet man in Python die Funktion `input()`.

Wenn ihr den folgenden Code ausführt, öffnet sich in Visual Studio Code ein Eingabefeld. Dort könnt ihr eine Eingabe tätigen und mit der Enter-Taste bestätigen.

```
eingabe = input('Bitte gib deinen Namen ein: ')
print(f'{eingabe}? Toller Name! Viel Spaß beim Programmieren lernen! :-)')
```

Wichtig ist: Eine Eingabe mit `input()` ist in Python **immer vom Datentyp `str`**. Das gilt auch dann, wenn eine Zahl eingegeben wird.

Wenn mit einer Eingabe gerechnet werden soll, muss sie zuerst in einen passenden Datentyp umgewandelt werden.

Konvertierung von Datentypen

Die Umwandlung eines Datentyps in einen anderen Datentyp nennt man **Konvertierung**. Manchmal wird auch der Begriff **Typumwandlung** verwendet.

Mit Zeichenketten kann Python nicht automatisch wie mit Zahlen rechnen. Deshalb müssen Zahleneingaben häufig mit `int()` oder `float()` umgewandelt werden.

Konvertierung zu einer Zeichenkette:

```
variable = 5
string_variable = str(variable)
print(string_variable, type(string_variable))
```

Konvertierung zu einer ganzen Zahl:

```
variable = '2'
int_variable = int(variable)
print(int_variable, type(int_variable))
```

Konvertierung zu einer Gleitkommazahl:

```
variable = '1.1'
float_variable = float(variable)
print(float_variable, type(float_variable))
```

Konvertierung zu einem Wahrheitswert:

```
variable = 1
bool_variable = bool(variable)
print(bool_variable, type(bool_variable))
```

Nicht jede Konvertierung ist möglich. Die Zeichenkette `'Hallo'` kann zum Beispiel nicht sinnvoll in eine ganze Zahl umgewandelt werden.

Außerdem kann Python Zahlen und Zeichenketten nicht einfach mit `+` zusammenfügen. Eine Zahl muss vorher mit `str()` in eine Zeichenkette umgewandelt werden. Alternativ kann man einen F-String verwenden.

```
a = 1.2

print('Ausgabe:', a)           # funktioniert
print('Ausgabe: ' + str(a))    # funktioniert
print(f'Ausgabe: {a}')         # funktioniert

# print('Ausgabe: ' + a)       # würde einen Fehler erzeugen
```

3.4. Rechenoperationen und Operatoren

Operatoren sind Zeichen oder Wörter, mit denen in Programmen Berechnungen, Vergleiche oder logische Verknüpfungen durchgeführt werden.

Operator	Bezeichnung	Beispiel	Hinweis
<code>+</code>	Addition	<code>zahl = a + b</code>	Bei Zeichenketten verbindet <code>+</code> zwei Zeichenketten.
<code>-</code>	Subtraktion	<code>ergebnis = a - b</code>	-
<code>*</code>	Multiplikation	<code>ergebnis = a * b</code>	Bei Zeichenketten wiederholt <code>*</code> eine Zeichenkette, zum Beispiel <code>'a' * 4</code> .
<code>/</code>	Division	<code>ergebnis = a / b</code>	Das Ergebnis einer Division ist in Python immer vom Typ <code>float</code> .
<code>//</code>	ganzzahlige Division	<code>ergebnis = a // b</code>	Gibt an, wie oft sich eine Zahl ganzzahlig teilen lässt.
<code>%</code>	Modulo / Rest der Division	<code>ergebnis = a % b</code>	Gibt den Rest bei einer ganzzahligen Division zurück.
<code>**</code>	Potenz	<code>quadrat = a ** 2</code>	Berechnet eine Potenz.
<code>or</code>	logisches ODER	<code>a or b</code>	Ergibt <code>True</code> , wenn mindestens einer der Werte <code>True</code> ist.
<code>and</code>	logisches UND	<code>a and b</code>	Ergibt <code>True</code> , wenn beide Werte <code>True</code> sind.
<code>not</code>	logisches NICHT	<code>not a</code>	Kehrt einen Wahrheitswert um.
<code><</code>	kleiner als	<code>a < b</code>	Gibt <code>True</code> zurück, wenn der Vergleich zutrifft.
<code><=</code>	kleiner gleich	<code>a <= b</code>	Gibt <code>True</code> zurück, wenn der Vergleich zutrifft.
<code>></code>	größer als	<code>a > b</code>	Gibt <code>True</code> zurück, wenn der Vergleich zutrifft.
<code>>=</code>	größer gleich	<code>a >= b</code>	Gibt <code>True</code> zurück, wenn der Vergleich zutrifft.
<code>!=</code>	ungleich	<code>a != b</code>	Gibt <code>True</code> zurück, wenn die Werte verschieden sind.
<code>==</code>	gleich	<code>a == b</code>	Gibt <code>True</code> zurück, wenn die Werte gleich sind.

Hinweis zum Schreibstil

In Python ist es üblich, links und rechts von Operatoren ein Leerzeichen zu setzen. Das erhöht die Lesbarkeit.

Schlechter lesbar: `ergebnis=a+b`

Besser lesbar: `ergebnis = a + b`



